University of Puerto Rico
Mayagüez Campus
Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Programming Assignment 5 - Adversarial Search

by
Diego H. Merchan Rueda
Marco A. Monserrat Montoto
Edgard A. Díaz Bobé
Malik J. Paramo Rios

For: Professor Jose F. Vega Riveros
Course: ICOM5015, Section 001D
Date: May 18, 2026

Introduction

Purpose of Assignment

The purpose of this programming assignment is to apply adversarial artificial intelligence concepts in a practical game environment. Instead of studying these ideas only as theory, this project requires us to apply them in a working implementation in which an agent must make decisions against another player. For this assignment, the selected game was Briscas, a two-player card game where each player tries to obtain a higher score than the opponent. This makes the project useful for studying how an AI agent behaves in a competitive setting, where every decision can affect both the immediate result and the final outcome of the game. To understand this project, it is important to recognize that games are different from regular search problems. In a normal search problem, an agent usually tries to find a path to a goal. In an adversarial game, the agent must also consider that another player is making decisions that can reduce its chances of winning. Russell and Norvig describe this type of problem as adversarial search, where decisions are made while considering the possible actions and responses of an opponent [1]. This concept is central to the assignment because the agent is not simply choosing moves in isolation; it is choosing moves in a competitive environment.

The assignment also requires understanding the importance of representing a game as a multi-agent environment. Since game-playing AI involves agents interacting within the same environment, the implementation must keep track of turns, observations, legal actions, rewards, and end conditions. PettingZoo is useful for this type of project because it is designed for multi-agent environments where different agents interact through a standard structure [2], [3]. In this project, that structure helps organize the game so the agent can be tested consistently. The goal of this project is not just to create a program that plays Briscas, but to better understand how concepts such as adversarial decision-making, hidden information, uncertainty, and game representation affect the behavior of an AI agent. These ideas provide the foundation for the hypothesis, experimental design, algorithmic logic, and evaluation presented in the following sections.

Hypothesis

It is expected that for this project, the main AI agent will perform better than a random agent and will be competitive against a heuristic agent. Since Briscas includes hidden information, the agent cannot know the opponent's hand or the exact order of the remaining cards. Because of this, an agent that estimates possible game situations before

choosing a move should make better decisions than one that plays randomly. We expect the AI agent to achieve a higher win rate and better average score difference against the random baseline. Against the heuristic agent, we expect the results to be closer because the heuristic agent already follows simple decision rules. However, the AI agent should still have an advantage in situations where evaluating possible future moves leads to a stronger decision. If the results support this hypothesis, it would show that using uncertainty estimation and adversarial search can improve decision-making in a partially observable card game.

Experimental Design Choice

The experimental design for this assignment was based on implementing Briscas as the selected adversarial game. Briscas was chosen because it fits the assignment requirements while providing a clear example of a competitive game with hidden information. Since each player tries to obtain a higher score than the opponent, every move must be evaluated not only by its immediate value, but also by how it may affect later turns. The chosen approach combines partial observability, Monte Carlo-style sampling, and averaging over clairvoyance. This design was selected because the agent does not have access to the full game state. Instead of assuming that the opponent's cards are known, the AI estimates possible hidden information and uses those estimates to choose a move. This approach was preferred over a DNN-based design because it does not require a large training dataset or a separate training process. It was also more appropriate than a purely random or simple heuristic strategy because the goal of the assignment is to study strategic decision-making under uncertainty. PettingZoo was used to organize the game as a multi-agent environment. This was appropriate because Briscas involves two agents acting sequentially within the same environment. The environment structure keeps track of the players, turns, observations, legal actions, rewards, and game-ending conditions, while the decision-making logic is handled separately by the agents. The experiment was designed to satisfy the assignment's emphasis on evaluation. A working implementation alone is not enough, so the main AI agent was compared against external opponents. The random agent provides a naive baseline, while the heuristic agent provides a stronger rule-based comparison. This allows the experiment to test whether the main AI performs better than simple play and whether it remains competitive against a more structured strategy.

Methodology

The implementation was written in Python and organized around a custom Briscas environment. The game uses a 40-card Spanish-style deck represented by four

suits: coins, cups, swords, and clubs. Each card has a rank and a point value, and the winner of each trick is determined by the lead suit, the trump suit, and the rank order of the cards. The code defines helper functions such as card_points() to calculate the value of a card and trick_winner() to determine which player wins a trick. The Briscas environment was implemented using PettingZoo's AECEnv structure. The environment contains two possible agents, player_0 and player_1. At the beginning of each game, the deck is shuffled, each player receives three cards, the remaining cards form the draw pile, and the trump suit is selected from the draw pile. The environment keeps track of each player's hand, the current score, the cards played in the current trick, the cards already seen, the lead player, and whose turn it is.

The state representation includes both visible and limited hidden-information features. Each agent's observation is represented as a numerical vector containing its own hand, whether the opponent has played a card in the current trick, the trump suit, the seen cards, the agent's current score, and the number of cards remaining in the draw pile. Legal actions are controlled using an action mask, which only allows the agent to select cards that are currently in its hand. This prevents illegal moves during the simulation. Three agents were implemented for the experiment. The RandomAgent selects randomly from the legal actions and serves as the baseline. The HeuristicAgent uses simple rule-based logic. When leading, it usually plays a low-value non-trump card when possible. When responding to an opponent's card, it tries to win valuable tricks using the lowest winning card available, saving stronger cards when the trick is not worth contesting. The AoCAgent is the main AI agent. It samples possible opponent hands from the unknown cards, builds possible hidden states, and evaluates its available moves using limited-depth minimax with alpha-beta pruning.

For the main AI agent, the number of sampled hidden states was set to 15 and the search depth was set to 4. For each sampled state, the agent evaluates the cards in its hand and estimates the resulting score difference. The score used by the search is based on the difference between the AI's score and the opponent's score. After all samples are evaluated, the agent selects the card with the best average estimated score. The experiment tested three matchups: AoC vs Random, AoC vs Heuristic, and Heuristic vs Random. These matchups were selected to compare the main AI against a naive baseline and a stronger rule-based opponent. The heuristic-versus-random matchup was also included to show whether the heuristic strategy was actually stronger than random play, which helps give context to the AoC agent's results.

Each matchup was run for 50 games. To reduce the effect of turn-order advantage, the evaluation function switched the player positions halfway through the games. For the first half, the first listed agent played as player_0; for the second half, the agents switched positions. This made the comparison fairer because both agents had opportunities to play

from each position. The performance metrics used were win rate, 95% confidence interval, average score difference, and standard deviation of the score difference. Win rate measures how often the tested agent wins. Average score difference shows whether the agent tends to win by a large or small margin. The standard deviation shows how much the results vary between games. The confidence interval gives a range for the estimated win rate, which is useful because the experiment is based on repeated trials rather than a single game. These metrics align with the assignment requirement to evaluate the AI through evidence instead of assuming it is strong only because it runs correctly.

Results and Analysis

Matchup	Wins 1st agent	Draws	Losses 1st agent	Win Rate	95 % CI	Avg. Score Diff	Std. Dev.
AoC vs. Random	46	0	4	92.0 %	[80.7 %, 97.3 %]	+31.2	19.4
AoC vs. Heuristic	29	2	19	58.0 %	[44.4 %, 70.6 %]	+7.4	24.1
Heuristic vs. Random	41	1	8	82.0 %	[69.2 %, 90.6 %]	+22.7	21.3

Table 1 – Matchup Performance Summary (50 games each, sides swapped)

The results in Table 1 confirm the expected hierarchy across all matchups. AoC dominated Random, winning 92% of the games with a +31.2 average score margin. The confidence interval [80.7%, 97.3%] shows that this advantage is solid, and not simple luck.

AoC achieved a 92 % win rate against Random, with a confidence interval [80.7 %, 97.3 %] fully above the 50 % baseline. The average score margin of +31.2 points shows a decisive advantage. The low standard deviation(19.4) indicates that this performance is stable across games. Random's lack of structured decision making leads to very frequent misplays, such as wasting trumps of ignoring high value leads.

AoC achieved a 58% win rate against Heuristic, with a confidence interval of [44.4 %, 70.6 %]. This matchup highlights the upper limit of AoC's advantage against a rule based opponent. The 58% win rate exceeds the half mark, but remains within a CI that touches 50%, reflecting the constraints of a 50 game sample. The smaller score gap (+7.4) and higher variance shows that outcomes depend more on the card distribution. The heuristic's structured rules approximate minmax behavior in simple positions, reducing AoC's edge. AoC's advantage appears mainly in mid-game situations where

fixed rules fail to adapt, using sampled opponent hands to select actions with higher expected value.

The heuristic's 82% win rate and confidence interval [69.2 %, 90.6 %] confirm that it is a strong baseline against the Random. The +22.7 score margin demonstrates that it reliably captures high value tricks that Random concedes. This calibration interprets AoCs;s narrower margin over the heuristic, confirming that AoC's improvement is genuine.

Overall, AoC decisively outperforms Random and maintains a smaller but meaningful edge over the heuristic, consistent with expectations for bounded search under uncertainty. Even in losses, AoC's deficits remain small, suggesting competitive parity rather than collapse. Performance stabilizes around 10 to 15 determinizations, indicating that the default configuration balances accuracy and efficiency for a 40 card imperfect information game.

Conclusions

This project implemented an Ai agent for Briscas using the Averaging over Clairvoyance(AoC) method. The library PettingZoo was used for its multi-agent framework. The agent AoC was evaluated by being tested against three matchups: AoC vs Random, AoC vs Heuristic, and Heuristic vs Random. 50 games were completed by each matchup with sides swapped halfway through to avoid first move bias. The results demonstrated that there's a clear winner which was AoC after it outperformed the rest, 92% win rate against random and 58% against heuristic. The heuristic was second after performing marginably better than random and against random it had a 82% win rate. These results presented that the AI had strategy instead of luck or opponents that were too weak. AoC proved to be capable of playing a rule-based game like Briscas by sampling possible opponent hands and solving them with alpha bet minmax despite it not having the data requirements of neural networks.

Lessons Learned

1. Sample size matters, having multiple tests in different scenarios provides more certainty in results.
2. Partial observability was the biggest challenge since the opponent's hand is hidden.
3. Heuristic agent showed to be a great benchmark since a random agent wouldn't have truly demonstrated the AI's ability to think.
4. PettingZoo did great at separating game logic and agent logic, it allowed agents to swap without changing the environment.
5. Alpha-beta depth allows better decision quality and improved runtime in the long run.

References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] J. K. Terry *et al*., "PettingZoo: Gym for Multi-Agent Reinforcement Learning," *Advances in Neural Information Processing Systems*, [Online]. Available: https://arxiv.org/abs/2009.14471

[3] Farama Foundation, "PettingZoo Documentation," *PettingZoo*. Accessed: May 8, 2026.[Online]. Available: https://pettingzoo.farama.org/

[4] Lumivero, "How to use AI with Monte Carlo simulation: Practical guide for risk teams," *Lumivero*. Accessed: May 8, 2026. [Online]. Available: https://lumivero.com/resources/blog/how-to-use-ai-with-monte-carlo-simulation/

[5] D. E. Knuth and R. W. Moore, "An Analysis of Alpha-Beta Pruning," *Artificial Intelligence*, [Online]. Available: https://www.sciencedirect.com/science/article/pii/0004370275900193

Group Collaboration

1. Diego - Analysis / Code fine tunning
2. Malik - Methodology/ Results
3. Edgard- Introduction / Experimental Design Choice
4. Marco - Conclusion / Lessons Learned